1. Race Condition, Lock and Semaphore

5 marks

Description
1. Create shared data or a shared counter.
2. Allow multiple threads to update the shared data without protection.
3. Compare the expected result with the actual result.
4. Repeat execution to demonstrate timing-dependent behaviour.
5. Correct the race condition using a Lock or equivalent mutual-exclusion mechanism.
6. Implement a Semaphore-based version.
7. Use the same workload for the unprotected, Lock-based and Semaphore-based versions.
8. Measure execution time for each version.
9. Compare correctness and synchronization overhead.

```
import threading
import time

# ----------------------------
# Configuration
# ----------------------------
NUM_THREADS = 4
ITERATIONS = 10000

EXPECTED_RESULT = NUM_THREADS * ITERATIONS
```

```python
# ----------------------------
# 1. Unprotected Version
# ----------------------------
def run_without_lock():
    counter = [0]

    def worker():
        for _ in range(ITERATIONS):
            temp = counter[0]
            time.sleep(0.000001)    # Creates
timing-dependent race
            counter[0] = temp + 1

    threads = []

    start = time.perf_counter()

    for _ in range(NUM_THREADS):
        t = threading.Thread(target=worker)
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    end = time.perf_counter()

    return counter[0], end - start
```

```python
# ----------------------------
# 2. Lock Version
# ----------------------------
def run_with_lock():
    counter = [0]
    lock = threading.Lock()

    def worker():
        for _ in range(ITERATIONS):
            with lock:
                temp = counter[0]
                time.sleep(0.000001)
                counter[0] = temp + 1

    threads = []

    start = time.perf_counter()

    for _ in range(NUM_THREADS):
        t = threading.Thread(target=worker)
        threads.append(t)
        t.start()

    for t in threads:
        t.join()
```

```
    end = time.perf_counter()

    return counter[0], end - start


# ----------------------------
# 3. Semaphore Version
# ----------------------------
def run_with_semaphore():
    counter = [0]
    semaphore = threading.Semaphore(1)

    def worker():
        for _ in range(ITERATIONS):
            semaphore.acquire()

            try:
                temp = counter[0]
                time.sleep(0.000001)
                counter[0] = temp + 1
            finally:
                semaphore.release()

    threads = []

    start = time.perf_counter()

    for _ in range(NUM_THREADS):
```

```
        t = threading.Thread(target=worker)
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    end = time.perf_counter()

    return counter[0], end - start


# ----------------------------
# Main Program
# ----------------------------
print("=" * 60)
print("RACE CONDITION, LOCK AND SEMAPHORE")
print("=" * 60)

print("\nNumber of Threads :", NUM_THREADS)
print("Iterations/Thread :", ITERATIONS)
print("Expected Result   :", EXPECTED_RESULT)


# Run without protection
result1, time1 = run_without_lock()

print("\n--- WITHOUT LOCK ---")
```

```
print("Actual Result     :", result1)
print("Expected Result   :", EXPECTED_RESULT)
print("Correct            :", result1 ==
EXPECTED_RESULT)
print("Execution Time    :", round(time1, 6),
"seconds")


# Run with Lock
result2, time2 = run_with_lock()

print("\n--- WITH LOCK ---")
print("Actual Result     :", result2)
print("Expected Result   :", EXPECTED_RESULT)
print("Correct            :", result2 ==
EXPECTED_RESULT)
print("Execution Time    :", round(time2, 6),
"seconds")


# Run with Semaphore
result3, time3 = run_with_semaphore()

print("\n--- WITH SEMAPHORE ---")
print("Actual Result     :", result3)
print("Expected Result   :", EXPECTED_RESULT)
print("Correct            :", result3 ==
EXPECTED_RESULT)
```

```
print("Execution Time    :", round(time3, 6),
"seconds")


# ----------------------------
# Comparison
# ----------------------------
print("\n" + "=" * 60)
print("COMPARISON")
print("=" * 60)

print("\nVersion\t\t\tResult\t\tTime (seconds)")
print("-" * 60)

print("Without Lock\t\t", result1, "\t\t",
round(time1, 6))
print("With Lock\t\t", result2, "\t\t",
round(time2, 6))
print("With Semaphore\t\t", result3, "\t\t",
round(time3, 6))

print("\nExpected Result:", EXPECTED_RESULT)

print("\nObservation:")
print("1. Without synchronization, a race condition
may occur.")
print("2. Lock provides mutual exclusion and gives
the correct result.")
```

```
print("3. Semaphore(1) also provides mutual exclusion.")
print("4. Lock and Semaphore introduce synchronization overhead.")
```

```
PS D:\5thSEM KMRU\operting System\LAB WORK\LAB> python -u "d:\5thSEM KMRU\operting System\LAB WORK\LAB\race_condition.py"
============================================================
RACE CONDITION, LOCK AND SEMAPHORE
============================================================

Number of Threads : 4
Iterations/Thread : 10000
Expected Result   : 40000

--- WITHOUT LOCK ---
Actual Result     : 10002
Expected Result   : 40000
Correct           : False
Execution Time    : 5.797327 seconds

--- WITH LOCK ---
Actual Result     : 40000
Expected Result   : 40000
Correct           : True
Execution Time    : 23.837695 seconds

--- WITH SEMAPHORE ---
Actual Result     : 40000
Expected Result   : 40000
Correct           : True
Execution Time    : 24.416668 seconds

============================================================
COMPARISON
============================================================

Version               Result         Time (seconds)
------------------------------------------------------------
Without Lock           10002          5.797327
With Lock              40000          23.837695
With Semaphore         40000          24.416668

Expected Result: 40000
```

```
Observation:
1. Without synchronization, a race condition may occur.
2. Lock provides mutual exclusion and gives the correct result.
3. Semaphore(1) also provides mutual exclusion.
4. Lock and Semaphore introduce synchronization overhead.
PS D:\5thSEM KMRU\operting System\LAB WORK\LAB> python -u "d:\5thSEM KMRU\operting System\LAB WORK\LAB\race_condition.py"
============================================================
RACE CONDITION, LOCK AND SEMAPHORE
============================================================

Number of Threads : 4
Iterations/Thread : 10000
Expected Result   : 40000

--- WITHOUT LOCK ---
Actual Result     : 10004
Expected Result   : 40000
Correct            : False
Execution Time    : 5.506368 seconds

--- WITH LOCK ---
Actual Result     : 40000
Expected Result   : 40000
Correct            : True
Execution Time    : 23.543085 seconds

--- WITH SEMAPHORE ---
Actual Result     : 40000
Expected Result   : 40000
Correct            : True
Execution Time    : 23.770024 seconds

============================================================
COMPARISON
============================================================

Version                Result          Time (seconds)
------------------------------------------------------------
Without Lock            10004           5.506368
With Lock               40000           23.543085
With Semaphore          40000           23.770024
```

```
--- WITH SEMAPHORE ---
Actual Result     : 40000
Expected Result   : 40000
Correct           : True
Execution Time    : 23.770024 seconds

============================================================
COMPARISON
============================================================

Version                 Result          Time (seconds)
------------------------------------------------------------
Without Lock             10004           5.506368
With Lock                40000           23.543085
With Semaphore           40000           23.770024

Expected Result: 40000

Observation:
1. Without synchronization, a race condition may occur.
2. Lock provides mutual exclusion and gives the correct result.
3. Semaphore(1) also provides mutual exclusion.
4. Lock and Semaphore introduce synchronization overhead.
PS D:\5thSEM KMRU\operting System\LAB WORK\LAB>
```